

Reclaiming Exascale Capacity: The Economic Case for forkrun on Frontier

Executive Summary

As exascale systems like Frontier push GPU solvers to extreme speeds, CPU-based data preparation and post-processing pipelines remain bottlenecked by tools designed for a pre-NUMA era. On Frontier, GNU Parallel's single-threaded dispatcher leaves entire nodes operating at ~6% CPU utilization during data-prep phases while the GPUs sit idle waiting for input. This inefficiency wastes not only CPU cycles but also the far more expensive GPU resources and prevents other science teams from using the oversubscribed system.

forkrun is a NUMA-aware, contention-free parallelizer that replaces GNU Parallel and `xargs -P`. On Frontier it is expected to accelerate data-prep pipelines by **10x–1000x** while raising CPU utilization from ~6% to >95%. This directly reclaims massive amounts of wasted node-hours, increasing total scientific throughput without additional hardware.

The Hidden Cost of Data Prep on Exascale

Frontier is heavily oversubscribed. Every node-hour is a strictly constrained resource. Scientific campaigns routinely spend a significant fraction of their allocation simply preparing data (unzipping, filtering, reformatting, routing inputs to GPUs). Most users parallelize this work with GNU Parallel, whose single-threaded Perl dispatcher is oblivious to Frontier's deep NUMA topology (4 NUMA domains per 64-core Trento CPU).

The result is severe economic inefficiency:

When dispatching microsecond-scale tasks, GNU Parallel saturates one core while the remaining 63 CPUs — **and the expensive GPUs that depend on them** — sit largely idle. The node continues to consume full baseline power and facility resources while delivering only ~6% useful work. Meanwhile, other science teams are denied or delayed because Frontier's node hours are being wasted on inefficient data preparation.

Estimated Economic Impact

Frontier's fully-loaded operational cost (power, staff, facility, hardware amortization) is approximately **\$1M per day**. Even modest reductions in wasted data-prep time yield large returns:

Scenario	Data-prep share of allocation	Expected speedup on Frontier	Recovered capacity for actual science	Estimated annual value
Floor	5%	10x	~4.5%	\$15–18M/year
Moderate	15%	20x	~14.2%	\$45–55M/year
High	30%	30x	~29.0%	\$100-110M/year

These figures assume conservative speedup ranges based on Frontier's 64-core Trento CPUs and 4× NUMA domains. A short Director's Discretionary validation run would quantify the exact numbers for Frontier workloads.

The forkrun ROI: Efficiency, Throughput, and Accessibility

forkrun treats data flow as a physical system. Using born-local NUMA placement, lock-free claiming, and SIMD scanning, it eliminates dispatcher overhead and scales cleanly across all four NUMA domains on Frontier's Trento CPUs — achieving >200,000 batch dispatches per second versus ~500 for GNU Parallel.

This directly improves four key metrics:

1. **Cost per Unit Science** — Compresses multi-hour data-prep jobs into minutes, amortizing fixed node costs over far more useful output.
 2. **Total Scientific Output** — Reclaims oversubscribed node-hours and returns them to the allocation pool for actual simulation.
 3. **Expanding the Parallelizable Surface Area** — Users can parallelize arbitrary multi-step shell functions with zero `fork/exec` overhead (`frun my_func < inputs`).
 4. **Zero Refactoring Cost** — Drop-in replacement. Existing workflows require only changing the command name.
-

Proposed Next Steps

forkrun is currently an open-source (MIT) tool proven on both UMA and NUMA hardware. To bring it to production readiness on Frontier, I propose a targeted collaboration:

1. **Validation** — Use a Director's Discretionary allocation to run synthetic benchmarks on Frontier's Trento nodes and capture real NUMA telemetry (expected: near-zero cross-socket traffic).
 2. **Case Studies** — Partner with 2–3 existing OLCF user groups currently bottlenecked by `parallel` or `xargs` in their data-prep pipelines.
 3. **Rollout** — Quantify recovered node-hours from these studies to justify facility-level integration of forkrun into the OLCF software stack.
-

Contact / Source

Anthony Barone

BSc Geophysics (UC Berkeley) • MSc Geophysics (UT Austin — advised by Mrinal Sen)

Dandridge, TN (1 hour from ORNL) | anthonywbarone@gmail.com | (858) 735-2342

<https://github.com/jkool702/forkrun>

Background: Computational Geophysics & Inverse Theory